

Проект по курсу Сложность Вычислений 3-КНФ.

Соловьев Иван, Б05-426

Весна, 2026

Аннотация

В данном проекте будет рассмотрена задача распознавания выполнимости 3-CNF-SAT. В процессе написания работы будет построен алгоритм A , для которого будет доказано два следующих утверждения:

- (а) Он распознаёт выполнимость 3-КНФ
- (б) В случае $P = NP$ на выполнимых формулах он заканчивает работу за полиномиальное время.

В основе работы алгоритма будет использоваться метод универсального поиска Левина, совмещённый с алгоритмом Дэвиса-Патнема-Логемана-Лавленда (далее будем сокращать до DPLL). Помимо этого, будет проверена корректность алгоритма, а также будет приведена оценка его временной сложности.

План текста

1	Введение	2
2	Предварительное ознакомление с используемой терминологией	2
2.1	КНФ	3
2.2	Задача выполнимости	3
3	Алгоритм DPLL и его анализ	3
3.1	Описание алгоритма	3
3.2	Псевдокод	4
3.3	Корректность правил упрощения	4
3.4	Корректность и полнота алгоритма	5

4	Универсальный поиск Левина и его анализ	6
4.1	Псевдокод реализации алгоритма	6
4.2	Анализ временной сложности	6
5	Реализация	7
5.1	Архитектура программной реализации	7
5.2	Перечисление программ	8
5.3	Итеративный DPLL с ограничением итераций	8
6	Доказательства корректности и выводы	8
6.1	Доказательство утверждения (а)	8
6.2	Доказательство утверждения (б)	9
6.3	Выводы	9

1 Введение

Задачи вида CNF-SAT являются довольно распространёнными в компьютерных науках, в частности, в теории сложности. Их суть состоит в верификации выполнимости КНФ (обычно задающихся с некоторыми ограничениями, например кол-вом литералов в дизъюнктах).

Более того, SAT-задачи являются NP-полными по теореме Кука-Левина [1]. Благодаря этому факту мы можем утверждать, что при истинности тождества $P = NP$, для задачи 3-CNF-SAT существует полиномиальный алгоритм.

Сама по себе 3-CNF-SAT является довольно распространённым примером, когда речь заходит об NP-полных задачах. В данной же работе речь идёт о построении алгоритма A , такого что он правильно распознаёт выполнимость 3-КНФ и работает за полиномиальное время, если $P = NP$.

Для реализации будет использоваться метод универсального поиска Левина [1], который позволит использовать наилучший среди имеющихся алгоритмов, не зная его заранее. Реализация поиска как такового будет опираться на алгоритм DPLL [2, 3], который является классическим методом решения SAT-задач.

2 Предварительное ознакомление с используемой терминологией

Для начала, стоит напомнить(или же познакомить) читателя с используемыми определениями и теоремой.

2.1 КНФ

Вспомним часть определений, ранее введённых в курсе Математической Логики и Теории Алгоритмов.

Определение 2.1. *Литералом* называется переменная p или её отрицание $\neg p$, где $p \in \{0, 1\}$.

Определение 2.2. *Дизъюнктом* называется дизъюнкция литералов.

$$D := (l_1 \vee l_2 \vee \dots \vee l_n), n \in \mathbb{N}$$

Определение 2.3. *Конъюнктивной Нормальной Формой (или же КНФ)* называется конъюнкция дизъюнктов:

Определение 2.4. *3-КНФ* называется КНФ, в которой каждый дизъюнкт содержит ровно три литерала.

Определение 2.5. *Присваиванием* называется функция $\sigma: \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$. Присваивание σ *выполняет* формулу φ , если $\varphi(\sigma) = 1$.

2.2 Задача выполнимости

Помимо этого, стоит сформулировать решаемую в данной работе задачу и теорему Кука-Левина.

Определение 2.6 (Задача 3-CNF-SAT). Для формулы φ в 3-КНФ определить, существует ли присваивание σ , при котором $\varphi(\sigma) = 1$. Такое присваивание называется *выполняющим*, а формула — *выполнимой*.

Теорема 2.1 (Кука-Левина, [1]). SAT является NP-полной задачей. В частности, 3-CNF-SAT также является NP-полной задачей.

3 Алгоритм DPLL и его анализ

3.1 Описание алгоритма

Алгоритм DPLL [2, 3] является полным алгоритмом решения SAT задач, основывающимся на поиске с backtracking'ом и ограничением ветвей.

Фактически, в DPLL мы оперируем двумя правилами во время обхода бинарного дерева:

Правило единичного литерала (eng: Unit Propagation). Если в дизъюнкте остался ровно один незафиксированный литерал l , он обязан быть истинным. Тогда значение соответствующей этому литералу переменной фиксируется, а сама формула упрощается.

Правило чистого литерала (eng: Pure Literal Elimination). Если переменная x_i встречается в формуле только как x_i , либо же только как $\neg x_i$, её можно зафиксировать, подставив более выгодное значение.

3.2 Псевдокод

Algorithm 1 DPLL(φ, σ)

```

 $\varphi \leftarrow \text{UnitPropagate}(\varphi, \sigma)$ 
if  $\varphi$  содержит пустой дизъюнкт then
    return NOT_SAT
end if
 $\varphi \leftarrow \text{PureLiteralElim}(\varphi, \sigma)$ 
if  $\varphi = \emptyset$  then
    return (SAT,  $\sigma$ )
end if
Некоторым образом выбираем переменную  $x \in \varphi$ 
 $\text{res} \leftarrow \text{DPLL}(\varphi[x=1], \sigma \cup \{x=1\})$ 
if  $\text{res} \neq \text{NOT\_SAT}$  then
    return res
end if
return DPLL( $\varphi[x=0], \sigma \cup \{x=0\}$ )

```

3.3 Корректность правил упрощения

Прежде чем доказывать корректность самого алгоритма, установим корректность двух правил упрощения, на которых он базируется.

Лемма 3.1 (Корректность Unit Propagation). Пусть дизъюнкт D содержит ровно один литерал l . Тогда формула φ выполнима тогда и только тогда, когда выполнима формула $\varphi' = \varphi[l=1]$, полученная фиксацией $l = 1$.

Доказательство. ($\Rightarrow$) Пусть σ выполняет φ . Так как $D = (l) \in \varphi$ и $\varphi(\sigma) = 1$, то $l(\sigma) = 1$. Следовательно, σ выполняет и $\varphi' = \varphi[l=1]$.

($\Leftarrow$) Пусть σ' выполняет $\varphi[l=1]$. Доопределим σ' значением $l(\sigma') = 1$ и обозначим новое присваивание как σ . Тогда все дизъюнкты, содержавшие

l , выполнены, а остальные выполнены по предположению. Значит, σ выполняет φ . $\square$

Лемма 3.2 (Корректность Pure Literal Elimination). Пусть переменная x встречается в φ либо только как x , либо же только как $\neg x$. Тогда формула φ выполнима тогда и только тогда, когда выполнима формула φ' , полученная выгодной фиксацией x .

Доказательство. Без ограничения общности предположим, что $\neg x$ вообще не встречается в φ . Тогда положим $x = 1$.

($\Leftarrow$) Очевидно. Если $\varphi[x=1]$ выполнима, то выполнима и φ .

($\Rightarrow$) Пусть σ выполняет φ . Определим σ' , совпадающее с σ на всех переменных кроме x , и положим $\sigma'(x) = 1$. Поскольку x не встречается отрицательно, замена $\sigma(x)$ на 1 не может сделать ни один дизъюнкт ложным: дизъюнкты, не содержащие x , никак не затронуты этим изменением, а дизъюнкты, содержащие x , теперь выполнены литералом x . Следовательно, σ' выполняет φ . $\square$

3.4 Корректность и полнота алгоритма

Теорема 3.1 (Корректность алгоритма DPLL). Если $\text{DPLL}(\varphi, \emptyset)$ возвращает присваивание σ , то $\varphi(\sigma) = 1$.

Доказательство. Воспользуемся индукцией по глубине рекурсии.

База индукции: Если $\varphi = \emptyset$, то все дизъюнкты удовлетворены присваиванием σ по построению.

Переход индукции: Пользуясь разделом 3.3, можем утверждать, что Unit Propagation и Pure Literal Elimination сохраняют выполнимость φ . При ветвлении алгоритм выбирает одно из двух значений переменной x_i . Значит, если рекурсивный вызов DPLL вернул σ , то, по предположению индукции, $\varphi(\sigma) = 1$. $\square$

Теорема 3.2 (Полнота DPLL). Если φ выполнима, то $\text{DPLL}(\varphi, \emptyset)$ вернёт выполняющее присваивание σ .

Доказательство. Дерево поиска DPLL конечно, поскольку на каждом уровне фиксируется хотя бы одна переменная (конкретнее, $\leq 2^n$ вершин во всём графе). Если φ выполнима, то существует ветвь, соответствующая выполняющему присваиванию. Значит, алгоритм при обходе дерева обязательно найдёт эту ветвь. $\square$

4 Универсальный поиск Левина и его анализ

Метод универсального поиска Левина был предложен ещё в 1973 году [1]. Его идея состоит в следующем: вместо того чтобы запускать одну конкретную машину Тьюринга (в нашем случае алгоритм), перечисляются все возможные $M_1, M_2, M_3, \dots$ и каждая запускается в рамках специального расписания, называемого в англоязычной литературе *dovetailing*.

4.1 Псевдокод реализации алгоритма

Пусть $V(\varphi, \sigma)$ — верификатор, проверяющий за время $O(|\varphi|)$, что σ выполняет φ .

Algorithm 2 Универсальный поиск Левина (φ)

```
1: for  $t = 1, 2, 3, \dots$  do
2:   for  $i = 1$  to  $\min(t, |\{M_i\}|)$  do
3:     Запустить  $M_i$  на  $\varphi$  ровно  $t$  шагов
4:     if  $M_i$  вернула  $\sigma \wedge V(\varphi, \sigma) = 1$  then
5:       return  $\sigma$ 
6:     end if
7:   end for
8:   Запустить  $M_F$  на  $\varphi$  с ограничением  $t^2 \cdot n$ 
9:   if  $M_F$  вернул NOT_SAT then
10:    return NOT_SAT
11:  end if
12:  if  $M_F$  вернул  $\sigma \wedge V(\varphi, \sigma) = 1$  then
13:    return  $\sigma$ 
14:  end if
15: end for
```

Замечание 4.1. МТ M_i , запускаемые с ограничением t шагов, способны доказать невыполнимость φ , но для этого может потребоваться довольно много итераций по t . По этой причине, для большего удобства вводится так называемая *fallback-машина*, обозначаемая как M_F , с ограничением $t^2 \cdot n$.

4.2 Анализ временной сложности

Теорема 4.1 (Условная полиномиальность). Если $P = NP$, то существует $k \in \mathbb{N}$ такое, что алгоритм A завершает работу на выполнимых формулах за время $O(|\varphi|^k)$.

Доказательство. Введём обозначение $n := |\varphi|$.

Предположим $P = NP$. Тогда существует машина Тьюринга M_{i^*} с индексом i^* в перечислении, решающая 3-CNF-SAT за время $T(n) = O(n^k)$ для некоторого $k \in \mathbb{N}$.

На итерации $t = T(n)$ машина M_{i^*} получает ровно $T(n)$ шагов и завершает работу, возвращая выполняющее присваивание σ . Верификатор подтверждает корректность σ за время $O(|\varphi|)$.

Тогда суммарное число шагов до наступления раунда $t = T(n)$ можно оценить следующим образом:

$$\sum_{t=1}^{T(n)} t \cdot \min(t, i^*) \leq i^* \cdot \sum_{t=1}^{T(n)} t = i^* \cdot \frac{T(n)(T(n) + 1)}{2} = O(i^* \cdot T(n)^2).$$

Поскольку i^* не зависит от n , получаем $O(T(n)^2) = O(n^{2k})$. $\square$

5 Реализация

5.1 Архитектура программной реализации

Реализация выполнена на C++20 и состоит из четырёх модулей.

В **Types** определены далее используемые типы (часть из них исключительно косметические).

DPLL реализует одноимённый алгоритм. Ключевые методы класса в этом модуле:

- **Simplify** — упрощение формулы при фиксации переменной;
- **UnitPropagate** — правило единичного литерала;
- **PureLiteralAssign** — правило чистого литерала;
- **ProcessAlgorithm(max_steps)** — итеративный DPLL с ограничением шагов, возвращающий SAT, NOT_SAT или TIMEOUT;
- **Verify** — верификатор присваивания;
- **BuildPrograms** — реализация паттерна "абстрактная фабрика" для MT(программ) $M_1, \dots, M_{24}$.

Levin реализует класс **UniversalSearch**. Метод **Solve** выполняет dovetailing: на итерации t запускает $M_1, \dots, M_{\min(t, 24)}$ за t шагов каждую, затем M_F за $t^2 \cdot n$ шагов.

Useful содержит вспомогательные функции: **Random3CNF** — генерация случайной 3-КНФ, **ParseDimacs** — разбор формата КНФ в формате DIMACS, **GetVariableCount** — подсчёт числа переменных.

5.2 Перечисление программ

В реализации используется конечное семейство из 24 программ: 4 детерминированные стратегии ветвления DPLL (минимальный индекс, максимальный индекс, наиболее частая переменная, наименее частая переменная) и 20 программ с seed'ами от 0 до 19 (конечно, это число можно увеличить, так как seed имеет тип `uint32_t`).

Теоретическая гарантия условной полиномиальности сохраняется, если M_{i*} принадлежит данному семейству или полиномиально эмулируется одной из стратегий.

Замечание 5.1. Под полиномиальной эмуляцией подразумевается существования такого полинома p , что для любого входа φ верно следующее неравенство: $T_{M_j}(\varphi) \leq p(T_{M_{i*}}(\varphi))$

5.3 Итеративный DPLL с ограничением итераций

Рекурсивная реализация DPLL не позволяет остановить алгоритм ровно через t шагов. Поэтому используется итеративная версия на стеке: каждое извлечение фрейма со стека считается одним шагом, и при превышении ограничения возвращается TIMEOUT.

6 Доказательства корректности и выводы

6.1 Доказательство утверждения (а)

Теорема 6.1 (Корректность алгоритма A). Алгоритм A распознаёт выполнимость 3-CNF-SAT:

- если A возвращает присваивание σ , то $\varphi(\sigma) = 1$;
- если A возвращает NOT_SAT, то φ невыполнима.

Доказательство. (Случай SAT) Алгоритм A возвращает присваивание σ только после того, как верификатор $V(\varphi, \sigma) = 1$. По определению верификатора это означает $\varphi(\sigma) = 1$.

(Случай NOT_SAT) Алгоритм A возвращает NOT_SAT только если fallback-машина с ограничением $t^2 \cdot n$ исчерпала всё дерево поиска и не нашла выполняющего присваивания. По теореме о полноте DPLL это происходит лишь в том случае, когда φ действительно невыполнима. $\square$

6.2 Доказательство утверждения (б)

Утверждение (б) является прямым следствием теоремы 4.1.

Следствие 6.1. Если $P = NP$, то на выполнимых формулах φ алгоритм A завершает работу за время $O(|\varphi|^{2k})$, где k — показатель полиномиальной сложности оптимальной МТ M_{i^*} .

6.3 Выводы

В данной работе был построен алгоритм A , являющийся комбинацией универсального поиска Левина и алгоритма DPLL. Более того, были доказаны два утверждения о нём.

Список литературы

- [1] S. Arora и B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009 (цит. на с. 2, 3, 6).
- [2] M. Davis, G. Logemann и D. Loveland. «A Machine Program for Theorem Proving». В: *Communications of the ACM* 5.7 (1962), с. 394—397 (цит. на с. 2, 3).
- [3] M. Davis и H. Putnam. «A Computing Procedure for Quantification Theory». В: *Journal of the ACM* 7.3 (1960), с. 201—215 (цит. на с. 2, 3).